Lab Handout 1-C
Serial Peripheral Interface (SPI) Bring-Up: STM32 ↔ SX1276 Register Access

Capstone — Module 1: Radio Frequency (RF) Hardware Foundations

1 Objective

Establish reliable SPI communication between the STM32 Nucleo-L476RG and the SX1276-based RFM95W module. You will read the `RegVersion` register and receive the expected value `0x12`, then demonstrate write/read-back and radio mode transitions. This handout is the first point at which firmware actually touches the RF silicon.

2 Prerequisites

- Lab Handout 1-A complete (breakout physically assembled).
- Lab Handout 1-B complete (toolchain working, blinky verified).

3 Materials

- Assembled RFM95W breakout (from LH1-A).
- Nucleo-L476RG + USB cable (from LH1-B).
- Half-size breadboard, 8–10 male–female jumper wires.
- Logic analyzer (optional but strongly recommended for troubleshooting).

4 Wiring

RFM95W Pin	Function	Nucleo Pin	Arduino Hdr
`VIN`	+3.3 V supply	`3V3`	3V3
`GND`	Ground	`GND`	GND
`SCK`	SPI clock	`PA5`	D13
`MISO`	SPI data in (to Microcontroller Unit (MCU))	`PA6`	D12
`MOSI`	SPI data out (from MCU)	`PA7`	D11
`CS`	Chip select (active low)	`PB6`	D10
`RST`	Reset (active low)	`PA9`	D8
`G0 (DIO0)`	Packet-done interrupt	`PA8`	D7

[Figure placeholder]

`LH1C_wiring_diagram.png`

Schematic of the Nucleo-L476RG ↔ RFM95W wiring. Power, ground, full-duplex SPI1, chip-select, reset, and `DIO0` interrupt line are all shown.

Figure 1: Schematic of the Nucleo-L476RG ↔ RFM95W wiring. Power, ground, full-duplex SPI1, chip-select, reset, and `DIO0` interrupt line are all shown.

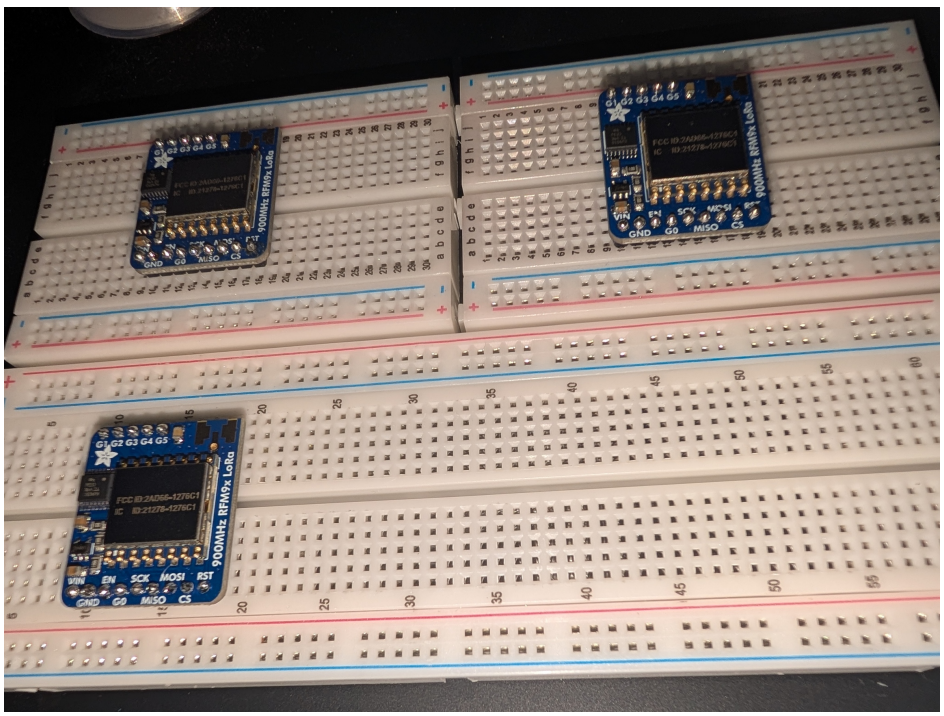

Figure 2: Physical breadboard build matching the schematic above. Keep jumper leads short to minimize SPI ringing.

Wire Before You Power

Double-check `VIN` vs. `GND`. The RFM95W will tolerate 3.3–5 V on `VIN` (it has an on-board regulator), but swapping `VIN` and `GND` *will* destroy the module. Verify with a DMM before plugging the Nucleo into USB.

5 Procedure

5.1 Step 1 — Configure peripherals via STM32CubeMX

1. Open the project from Lab Handout 1-B (or create a new one if you prefer to keep blinky separate). Open the CubeMX perspective.
2. Enable **SPI1** in **Full-Duplex Master** mode.
3. Configure SPI1 with the settings in Table 1.
4. Configure `PB6`, `PA9`, `PA8` as GPIO outputs named `RFM_CS`, `RFM_RST`, `RFM_DIO0` (`DIO0` will become an `EXTI` input later; leave as plain GPIO input for this handout).
5. Enable **USART2** in asynchronous mode at 115200 8N1 (for `printf` redirection to the ST-LINK virtual COM port).
6. Generate code.

Table 1: SPI1 configuration for RFM95W register access.

Parameter	**Value**
Mode	Full-Duplex Master
Data size	8 bits
First bit	MSB first
Prescaler	Set so that SPI clock $\leq$5 MHz (safe margin below the 10 MHz max)
CPOL	0 (Low)
CPHA	1 Edge
NSS	Software

[Figure placeholder]

`LH1C_spi_config.png`

STM32CubeMX SPI1 parameter panel configured per Table 1: 8-bit data, MSB first, CPOL=0, CPHA=1 Edge, prescaler chosen for $\leq$5 MHz clock.

Figure 3: STM32CubeMX SPI1 parameter panel configured per Table 1: 8-bit data, MSB first, CPOL=0, CPHA=1 Edge, prescaler chosen for $\leq$5 MHz clock.

5.2 Step 2 — Redirect `printf` to USART2

1. In `main.c`, add an override for `__io_putchar` that calls `HAL_UART_Transmit(&huart2, ...)`. A standard snippet:

   ```
   int __io_putchar(int ch) {
       HAL_UART_Transmit(&huart2, (uint8_t*)&ch, 1, HAL_MAX_DELAY);
       return ch;
   }
   ```

2. In the build configuration, ensure the `-specs=nano.specs` and `-u _printf_float` flags (or equivalent) are set so the reduced-newlib `printf` links.

5.3 Step 3 — Write a minimal register-read function

Implement the following helper:

```
uint8_t rfm_read_reg(uint8_t addr) {
    uint8_t tx[2] = { addr & 0x7F, 0x00 };  // bit 7 = 0 => read
    uint8_t rx[2] = { 0, 0 };
    HAL_GPIO_WritePin(RFM_CS_GPIO_Port, RFM_CS_Pin, GPIO_PIN_RESET);
    HAL_SPI_TransmitReceive(&hspi1, tx, rx, 2, HAL_MAX_DELAY);
    HAL_GPIO_WritePin(RFM_CS_GPIO_Port, RFM_CS_Pin, GPIO_PIN_SET);
    return rx[1];
}
```

And the matching write helper (bit $7 = 1$, followed by the value byte).

5.4 Step 4 — Reset the module and read `RegVersion`

1. In `main` after Hardware Abstraction Layer (HAL) init, drive `RFM_RST` low for at least 100 μs, then high, then wait 10 ms for the module to come out of reset.
2. Call `rfm_read_reg(0x42)` and `printf` the result.
3. Flash and run. Open the serial monitor (115200,8N1).
4. You should see `RegVersion = 0x12`. Any other value indicates a wiring or configuration error.

Why 0x12?

The SX1276 silicon revision register returns `0x12` for the V1b mask set used in the RFM9x breakouts. This is the single most reliable sanity check for SPI wiring, clock polarity, and reset sequencing: if you read `0x00` or `0xFF`, the module is not talking to you; if you read anything else, you have a clock phase or byte-order issue.

[Figure placeholder]

`LH1C_regversion_printf.png`

Serial monitor output showing the first successful `RegVersion = 0x12` read. This is the canonical sanity check for the entire SPI stack.

Figure 4: Serial monitor output showing the first successful `RegVersion = 0x12` read. This is the canonical sanity check for the entire SPI stack.

5.5 Step 5 — Register write / read-back

1. Write the sequence `0xD9 0x0C 0xCC` to `RegFrfMsb`, `RegFrfMid`, `RegFrfLsb` (address `0x06`–`0x08`). This sets the carrier frequency to 906.5 MHz. Note: the module must be in `SLEEP` mode before frequency writes take effect.
2. Read the same registers back. Verify that all three values match what you wrote.

5.6 Step 6 — Mode transitions

1. Read `RegOpMode` (`0x01`). Confirm the low 3 bits reflect the current mode.
2. Write `0x80` to `RegOpMode` (LoRa mode, `SLEEP`).
3. Write `0x81` (LoRa mode, `STANDBY`). Read back and confirm.
4. Write `0x83` (LoRa mode, `TX`). Read back and confirm. The module will return to `STANDBY` after any transmission attempt; this is expected.

6 Verification Checklist

- ☐ `RegVersion` reads `0x12`.
- ☐ Write/read-back to `RegFrf*` registers returns identical values.
- ☐ `RegOpMode` mode bits transition correctly through `SLEEP`, `STANDBY`, `TX`.
- ☐ Serial output shows all register values in hex over the ST-LINK virtual COM port.

Troubleshooting

- `RegVersion` **reads** `0x00`: `MISO` not connected, or `CS` never asserted.
- `RegVersion` **reads** `0xFF`: `MISO` floating (pull-up). Check the jumper to `PA6`.
- **Reads a seemingly-random but repeatable value**: CPOL/CPHA wrong, or bit 7 (read/write) flipped in your address byte.
- **First read is wrong, subsequent reads correct**: missing reset delay; increase the post-reset wait to 20 ms.

Lab Handout 1-C Deliverable

1. Source code for your `rfm_read_reg` / `rfm_write_reg` helpers and the `main` test sequence.
2. Serial monitor log showing `RegVersion = 0x12`, frequency register write/read-back, and all three mode transitions.
3. If a logic analyzer was used: one screenshot of a successful `RegVersion` read transaction with `CS`, `SCK`, `MOSI`, `MISO` traces.

Glossary

HAL	Hardware Abstraction Layer	5
MCU	Microcontroller Unit	1
RF	Radio Frequency	1
SPI	Serial Peripheral Interface	1, 2, 3, 5